

MP1DS 2022/2023

Développement des applications répartis

RPC

Programmation réseau sous Unix

- ✓ L'utilisateur ne peut accéder directement aux protocoles situés dans le noyau d'Unix
 - ⇒ Il peut utiliser les protocoles TCP et UDP par le biais des sockets
 - ⇒ Les sockets peuvent être vues comme des portes vers le réseau
 - ⇒ Les sockets offrent, pour la programmation, une interface similaire à celle du système de fichiers
- ✓ Le programmeur peut accéder directement aux points d'entrées de la couche 4 que forment les sockets sans avoir à traverser les couches 5 et 6
 - ⇒ Ce qui est contraire aux règles imposées par le modèle de référence OSI de l'ISO

RPC?

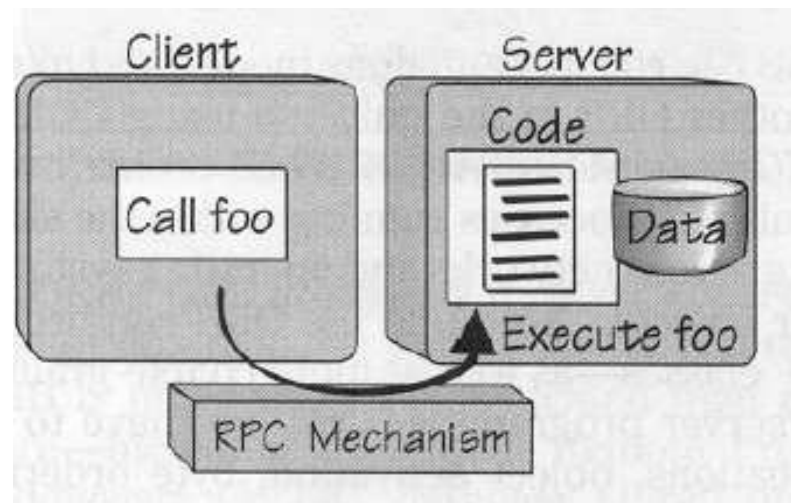
✓ RPC -- Remote procedure call?

⇒ Appel de procédures/fonctions à distance

⇒ Abstraction:

↳ Vis-à-vis du protocole de communication

↳ Distribution masquée (transparence)



Composants de RPC?

- ⇒ **IDL**-- Interface Definition Language: langage de description des données et des fonctions proche du C;
- ⇒ une méthode de représentation portable des données;
- ⇒ un protocole d'appel :
 - ↳ transmission des paramètres de l'appel du client vers le serveur;
 - ↳ transmission du résultat du serveur vers le client;
- ⇒ un protocole de publication et de recherche d'un service.

Types RPC

✓ **RPC par host-serveur**: le client doit spécifier le host-serveur.

⇒ Deux grandes normes (traditionnelles):

↳ **SUN ONC/RPC** --Open Network Computing/Remote Procedure Call

↳ **OSF DCE** -- Open Software Foundation - Distributed Computing Environment.

✓ **Mais aussi RPC par objet-serveur** : le client ne désigne pas le host-serveur

⇒ **OMG CORBA** -- Object Management Group -Common Object Request Broker Architecture

⇒ **SUN Java RMI** -- Remote Method Invocation

⇒ **Microsoft - DCOM** -- Distributed Component Object Model

Types RPC (suite)

✓ Et RPC intégrées dans les systèmes de composants:

- ⇒ SUN J2EE EJB -- Java 2 (Platform) Enterprise Edition - Enterprise Java
- ⇒ OMG CCM -- Object Management Group - Corba Component Model
- ⇒ WS-SOAP -- Web Services - Simple Object Access Protocol

ONC RPC (SUN)

- ✓ rfc 1831
- ✓ Solution classique, disponible sur Unix (incluse dans le système en général) et sous windows;
- ✓ « Open source », comparable aux DCE RPC en "gratuit"
- ✓ Il utilise le protocole **XDR** pour la description/échange de données (transport des arguments et du résultat) -- (rfc 1832) ;
- ✓ Technologie utilisée par NFS--*Network File System*.
- ✓ Outil de base **rpcgen**:
 - ⇒ génère des convertisseurs ((un)marshalling);
 - ⇒ génère les *souches* (serveurs et clients);
 - ⇒ propose des modèles pour le client et le serveur engendre un Makefile;
- ✓ Utilise un **lookup service** pour la publication et la découverte d'un service (rfc 1833);
 - ⇒ *Service lookup tourne sur chaque hôte contenant des serveurs rpc.*
- ✓ Les RPC ONC peuvent être implantées sur n'importe quelle couche transport (TCP/UDP)

Représentation de Données (en mémoire)

✓ Client et serveur sur machines différentes

⇒ Il faut transmettre données et résultats

✓ Différences de représentations en mémoire

⇒ Codage Little-endian (x86, alpha) vs. Big-endian (ppc, sparc, reseau) :
(depend du processeur)



⇒ Représentation des flottants (fixe par IEEE 754)

⇒ Alignement des données dans structure (dépend du compilateur)

⇒ Tailles (32 ou 64 bits ; 4 ou 8 octets pour un registre)

type	char	short	int	long	long long	*	float	double
sizeof	1	2	4	4/8	8	4/8	4	8

XDR -eXchange Data Representation

- ✓ Représentation portable des données (réseau):
 - ⇒ codage *big endian* avec des mots de 4 octets (complétion par des zéros)
 - ⇒ permet de définir des **types de données associés à une représentation binaire standard**
 - ⇒ Représentation commune sur le réseau (format pivot)
 - ⇒ orienté C (syntaxe proche du C):
 - ↳ types fondamentaux classiques (int, double, etc.)
 - ↳ énumération
 - ↳ struct et union
 - ↳ tableaux
 - ⇒ rpcgen engendre des fonctions de traduction à partir d'un fichier XDR (et des bibliothèques de conversion de bas niveau) en XDR

XDR -Types (1/2)

Type	Identificateur	taille
Entier relatif	int	4 octets (IEEE)
Entier positif	unsigned int	4 octets (IEEE)
Enumération	enum {id = constante, ...}	4 octets (IEEE)
Booléen	bool	Enum {False=0, True=1}
Entier long	hyper	8 octets
Entier positif	unsigend hyper	8 octets

Type	Identificateur	taille
Réels simples	Float	4 octets (IEEE)
Réels doubles	double	4 octets (IEEE)
Réels long	quadruple	4 octets (IEEE)
Rien	void	0 octet

XDR -Types (2/2)

✓ Données à taille fixe ou variable :

⇒ [n] correspond à une taille fixe égale à n

⇒ <> correspond à une taille variable

⇒ <m> correspond à une taille variable inférieure à m

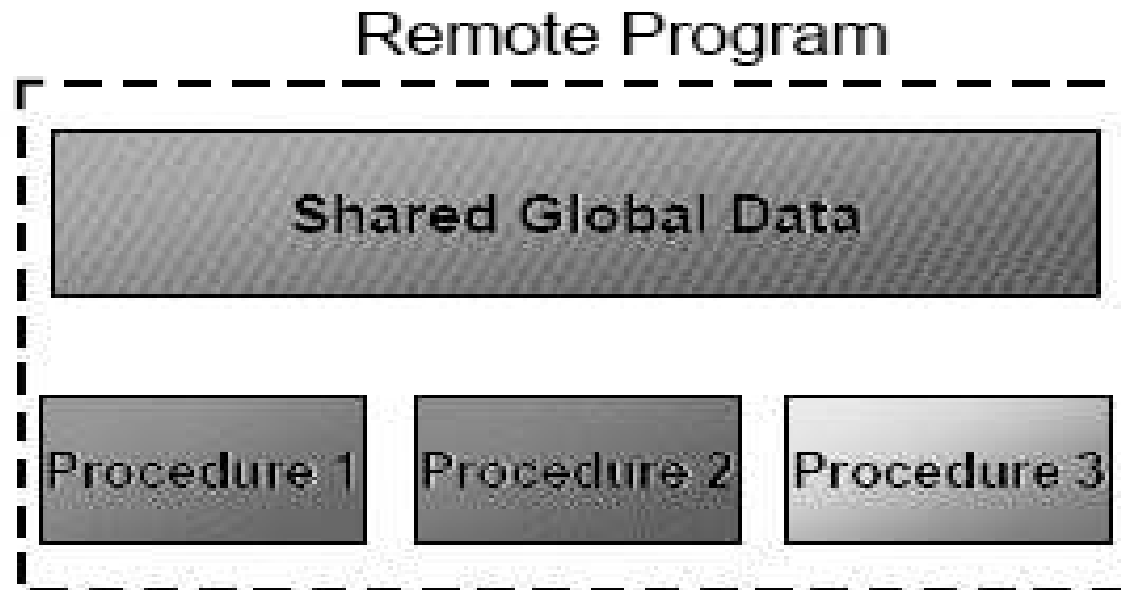
⇒ exemple, int<15> : liste d'entiers de longueur au plus 15.

Cas particuliers :

Identificateur	Contenu
opaque	Liste d'octets
string	Chaîne de caractères

Principe de base de SUN RPC

- ✓ Un service réseau est fourni par plusieurs programmes
- ✓ Un programme fournis plusieurs fonctions (ou procédures)
- ✓ Un client appelle une procédure d'un programme



Le protocole RPC --Identification

✓ Une procédure est identifiée par trois entiers (positifs) :

- ① Hostname (IP Address)
- ② Le numéro de programme unique (32 bit integer)

↳ Sun divise les IDs en :

0x00000000 - 0x1fffffff	Serveurs officiels Sun
0x20000000 - 0x3fffffff	Publique (Admin. système)
0x40000000 - 0x5fffffff	Serveurs Transitoires
0x60000000 - 0xffffffff	Réservée par Sun

⇒ Le numéro de procédure (32 bit integer)

↳ Commence toujours à partir de 1 et puis incrémenter séquentiellement.

- ③ Le numéro de version du programme (pour tests et migration)

↳ Commence typiquement à partir de 1 et puis incrémenter

Le Langage de Description RPCL (1/2)

- ✓ IDL de RPC
- ✓ Les fichiers spécification d'extension ".x" ont la structure suivante:

[Définition des constantes]

[Définition des types]

program NOM_PROGRAMME {

 [**version** NOM_VERSION {

 [type_resultat nom_procedure

 (type_du_parametre) = **numero_procedure**;

 } = **numero_version**]

} = **numero_programme**;

Le Langage de Description RPCL (2/2)

- ✓ Les constantes :

`const identificateur = valeur;`

- ✓ Les structures :

```
struct nom_du_type {  
    type attribut ;  
}
```

- ✓ Nouveau type:

`typedef type nom_du_type ;`

⇒ Cas particulier des tableaux et chaînes de caractères :

`typedef int vecteur <1000>`

`typedef string chaine <255>`

Liste des services RPC enregistrés sur une machine

- ✓ Liste des services enregistrés:

⇒ **rpcinfo -p [machine]**

- ✓ Appel de la procédure 0 d'un programme en utilisant le protocole udp

⇒ **rpcinfo -u machine num_prg [num_version]**

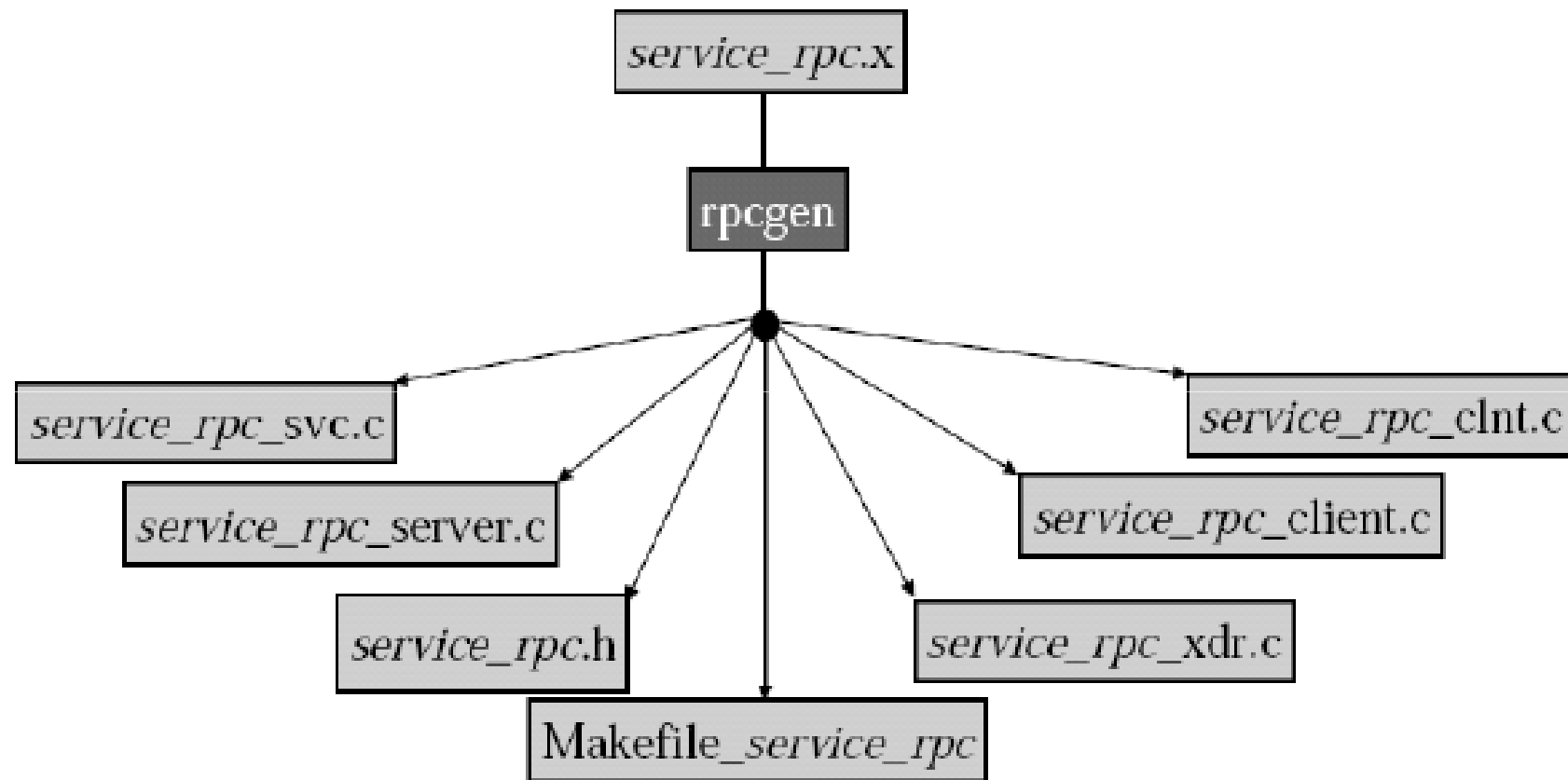
⇒ **rpcinfo -t machine num_prg [num_version]**

- ✓ Exemple: **rpcinfo -p**

program	no_version		protocole	no_port	
100000	2		tcp	111	portmapper
100000	2		udp	111	portmapper
100021	1	udp	32768	nlockmgr	
100021	3	udp	32768	nlockmgr	
100021	1	tcp	32768	nlockmgr	
100007	2	udp	705	ypbind	
100007	1	udp	705	ypbind	
100007	2	tcp	708	ypbind	
100007	1	tcp	708	ypbind	
100003	2	udp	2049	nfs	

RPC/RMI

RPCGEN



Exemple: Helloworld.x

- ✓ Le fichier de description helloworld.x

```
typedef string chaine<255>;  
program HELLO_WORLD_PROG {  
    version HELLO_WORLD_VERSION_1 {  
        void hello_world_null(void)=0; /* tester le réseau et le serveur; sorte de ping*/  
        chaine hello_world(chaine)=1;  
    }=1;  
} = 0x22222220;
```

- ✓ rpcgen helloworld.x

- ⇒ helloworld.h (fichier entête)
- ⇒ helloworld_xdr.c (filtre xdr)
- ⇒ helloworld_svc.c (souche serveur)
- ⇒ helloworld_clnt.c (souche client)

- ✓ rpcgen -a helloworld.x

- ⇒ Makefile
- ⇒ helloworld_server.c

helloworld_server.c

/ This is sample code generated by rpcgen . These are only templates and you can use them as a guideline for developing your own functions.*/*

```
#include "helloworld.h"
```

```
void * hello_world_null_1_svc(void *argp, struct svc_req *rqstp) {  
    static char * result;  
    printf ("Ping\n");fflush (stdout);  
    return (void *) &result;  
}
```

```
chaine * hello_world_1_svc(chaine *argp, struct svc_req *rqstp) {  
    static chaine result;  
    static char tab[255];  
    /* insert server code here */  
    result=tab;  
    strcpy (result, "Hello ");  
    strcat (result, *argp);  
    printf ("Result: %s\n", *argp);  
    printf ("Result: %s\n", result);  
    return &result;  
}
```

helloworld_client.c

```
#include "helloworld.h"

void hello_world_prog_1(char *host) {
    CLIENT *clnt;
    void *result_1;
    char *hello_world_null_1_arg;   chaine *result_2;   chaine hello_world_1_arg;
    /* On établit la connexion en spécifiant le host, le #prog, le #version et le transport utilisé */
    clnt = clnt_create (host, HELLO_WORLD_PROG, HELLO_WORLD_VERSION_1, "udp");
    if (clnt == NULL) { clnt_pcreateerror (host);      exit (1);  }
    result_1 = hello_world_null_1((void*)&hello_world_null_1_arg, clnt);
    if (result_1 == (void *) NULL) {
        clnt_perror (clnt, "call failed"); }
    result_2 = hello_world_1(&host, clnt);
    if (result_2 == (chaine *) NULL) clnt_perror (clnt, "call failed");
    printf ("%s\n", *result_2);
    clnt_destroy (clnt); /* on coupe la connexion*/
}

int main (int argc, char *argv[]) {
    if (argc < 2) {
        printf ("usage: %s server_host\n", argv[0]);
        exit (1); }
    hello_workd_prog_1(argv[1]);}
```

Création du serveur & client

- ✓ `rpcgen helloworld.x`
- ✓ `gcc -o server helloworld_server.c helloworld_xdr.c helloworld_svc.c -lrpcsvc -lnsl`
- ✓ `./serveur &`
- ✓ `./rpcinfo -p`
- ✓ `./rpcinfo -u num HELLO_WORLD_PROG 1`
- ✓ `gcc -o client helloworld_client.c helloworld_xdr.c helloworld_clnt.c -lnsl`
- ✓ `./client`

Publication et Découverte

- ✓ Un problème standard des systèmes distribués :
 - ⇒ pour le serveur, se faire connaître (**publication**)
 - ⇒ pour le client, trouver le serveur (**découverte et résolution**)
- ✓ Plusieurs niveaux :
 1. partie facile : nom de la machine du serveur + DNS connexion possible (TCP/UDP)
 2. quelle couche transport ?
 3. pour TCP/UDP, quel numéro de port ?
- ✓ Solution proposée par les RPC ONC, un programme de *binding* (RFC 1823) :
 - ⇒ le **portmapper** (*version 2*) : *spécifique à TCP/UDP*
 - ↳ le portmapper est lui-même un serveur rpc.
 - ⇒ **rpcbind** (*versions 3 et 4*) : *indépendant du transport*

Binding RPC

✓ Principes communs aux différentes versions:

⇒ le binder est un programme RPC de numéro 100000

⇒ il écoute toujours sur les ports 111 UDP et TCP

⇒ Le portmapper offre différentes fonctions :

⇒ publication d'un programme (décrit par son #programme et son #version)

⇒ suppression d'un programme

⇒ découverte (résolution) d'un programme, c'est-à-dire obtention du port associé à un programme

⇒ obtention de la liste des programmes enregistrés

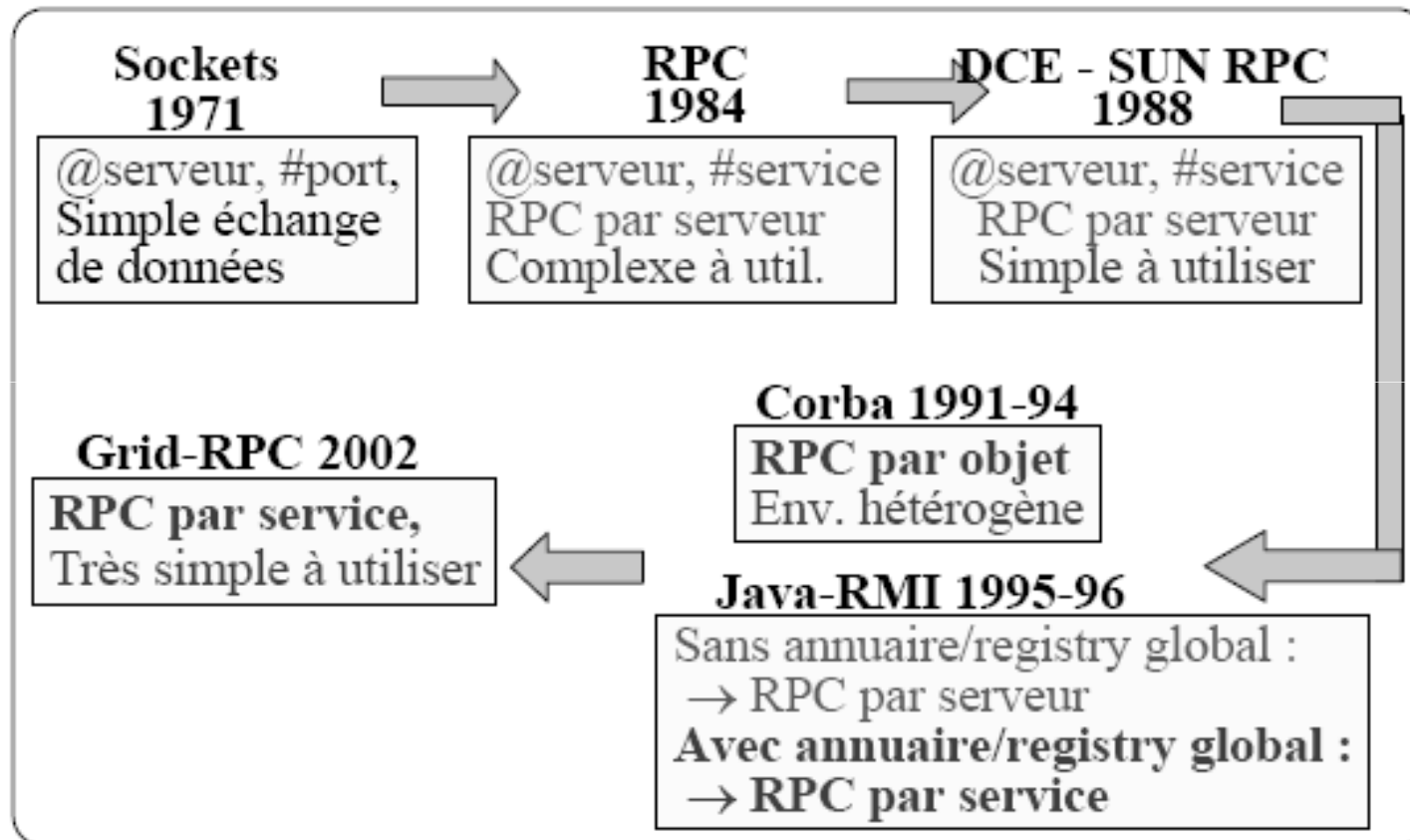
⇒ Appel direct d'une fonction distante

✓ Le portmapper est limité à TCP et UDP.

RPCbind

- ✓ Remplace le portmapper.
- ✓ Principaux avantages :
 - ⇒ indépendant du transport
 - ⇒ permet un enregistrement et une résolution incluant les informations de transport:
 - ⇒ offre quelques nouvelles fonctions :
 - ↳ horloge
 - ↳ statistiques d'utilisation
 - ↳ un programme peut enregistrer plusieurs instances correspondant à différents transport (avec le portmapper, le transport est automatique celui utilisé pour contacter le portmapper lui-même)

Conclusion -- Bilan des RPC



XDR?? CDR dans CORBA